
User:

as interviewer expected clearly concept and enough understanding for me

ChatGPT:

Great — let's lock in that "interviewer-expected" clarity and give you *enough understanding* to feel confident too. Just let me know the topic, keyword, or question you're working on, and I'll break it down like:

- **What it is** (simple & clear)
- **How it works**
- **Real-world analogy or example**
- **Common tricky part or misconception**
- **Interviewer Pro Tip** (what they might test or dig into)

You can shoot a topic like:

- "final vs finally vs finalize"
- "How HashMap works internally"
- "volatile keyword"
- "Object equality in Java"
- Or any DSA or code snippet too.

Drop the next one you want to master, and I'll tailor it just right.

User:

86. What are the key features of Java 8?

ChatGPT:

Perfect. Here's a crisp, interview-ready answer for:

86. What are the key features of Java 8?

✓ 1. Lambda Expressions

What: Anonymous functions that reduce boilerplate code, especially for functional interfaces.

Syntax:

```
java  
(a, b) -> a + b
```

Use: Simplifies operations like sorting, filtering in collections.

Pro Tip: Interviewers may ask: "*Can lambdas access local variables?*"
→ Yes, but they must be **effectively final**.

✓ 2. Functional Interfaces

What: Interfaces with a **single abstract method** (SAM).

Example: Runnable, Callable, Comparator, or custom ones with @FunctionalInterface.

```
java
@FunctionalInterface
interface MyFunc {
    int apply(int a, int b);
}
```

Common Q: "What happens if you add two abstract methods to a functional interface?"
→ It won't compile.

✓ 3. Stream API

What: Processes collections in a **declarative and functional style**.

```
java
list.stream().filter(e -> e > 10).map(e -> e * 2).collect(Collectors.toList());
```

Features:

- Lazy evaluation
- Parallel execution
- Chainable methods (like filter, map, reduce)

Trick: Stream does **not modify original data**; it creates a new result.

✓ 4. Default & Static Methods in Interfaces

What:

- default methods let you provide method bodies in interfaces.
- static methods can be utility methods in interfaces.

```
java
interface MyInterface {
    default void show() { System.out.println("Default"); }
    static void log() { System.out.println("Static"); }
}
```

Interviewer may ask: "What if two interfaces have the same default method?"
→ You **must override** it in your class to resolve ambiguity.

✓ 5. Method References

What: Shorthand for calling existing methods using ::.

```
java
list.forEach(System.out::println);           // Equivalent to: list.forEach(x -> System.out.println(x))
```

Types:

- Static: ClassName::staticMethod
 - Instance: object::instanceMethod
 - Constructor: ClassName::new
-

✓ 6. Optional Class

What: A container to avoid null checks and NullPointerException.

```
java
Optional<String> name = Optional.of("John");
name.ifPresent(System.out::println);
```

Pro Tip: Interviewers test this as an alternative to null checks.

✅ 7. New Date & Time API (java.time)

Why: Old Date and Calendar were **mutable**, **not thread-safe**, and **confusing**.

Key classes:

- LocalDate, LocalTime, LocalDateTime
- ZonedDateTime
- DateTimeFormatter

```
java
LocalDate today = LocalDate.now();
```

Analogy: Think of it like Joda-Time built into core Java.

✅ 8. Nashorn JavaScript Engine

Allows you to run JavaScript code inside Java.

```
java
ScriptEngine engine = new ScriptEngineManager().getEngineByName("nashorn");
engine.eval("print('Hello from JS');");
```

✅ 9. Collectors Utility (with Stream)

Like toList(), toSet(), joining(), groupingBy() to collect processed stream data.

✅ 10. Parallel Streams

stream().parallel() enables multicore processing of data.

Caution: Use only when thread safety and performance gains are guaranteed.

🧠 Summary for Interviews

If short on time, just remember:

"Java 8 introduced functional programming to Java via lambda expressions, functional interfaces, Stream API, default methods, and Optional. It also modernized date/time handling and enabled parallel & concise collection processing."

Want a **mock Q&A**, or a **short version** for quick revision too?
